

OFF-BY-ONE
2025

COM-pletely Unplanned

: A Windows Bug Hunter's Journey to LPE

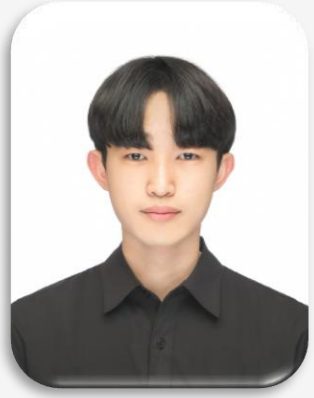
ENKI WHITEHAT

Jongseong Kim, Dongjun Kim

About us



- **Jongseong Kim**
  - ENKI WhiteHat, Security Researcher
  - Interested in Windows Offensive Security
  - Windows Bug Hunter



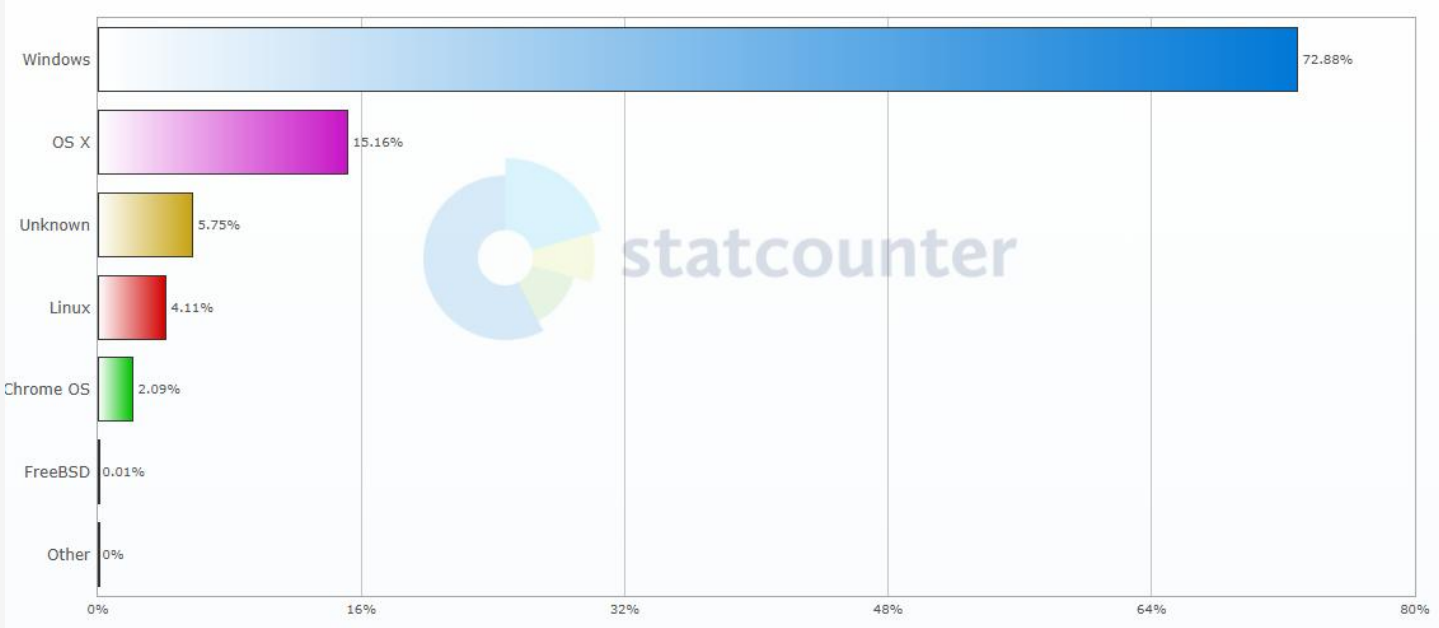
- **Dongjun Kim**
  - ENKI WhiteHat, Security Researcher
  - Interested in Windows, Browser Offensive Security
  - Windows Exploit Development

Agenda

1. Introduction
2. Backgrounds
3. Learning from the Past
4. Discovered Vulnerabilities
5. Results

Why Windows LPE?

- Windows is one of the most widely used OS
- Windows has a large codebase and complex structure
- **LPE** is a key threat in sandboxed environments.





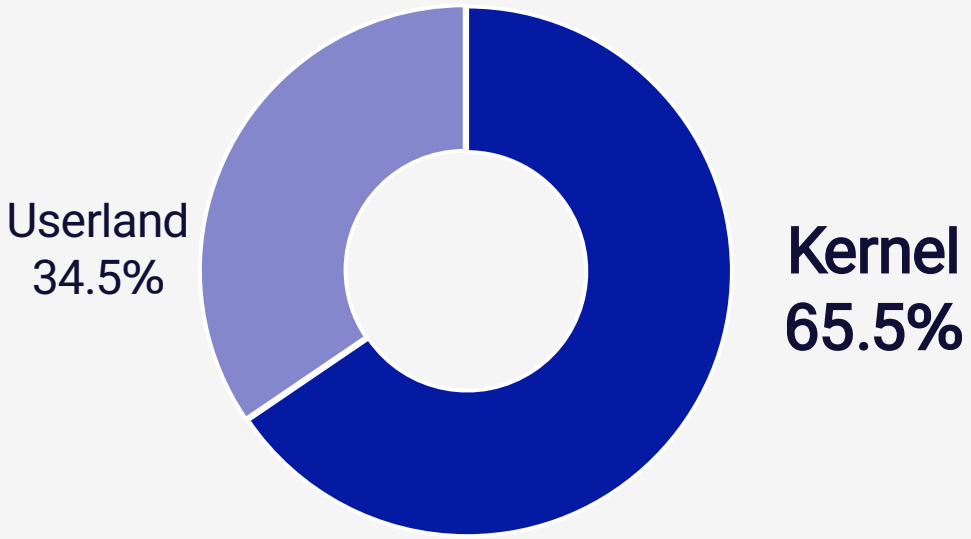
[1] https://gs.statcounter.com/os-market-share/desktop/worldwide/#monthly-202401-202412-bar

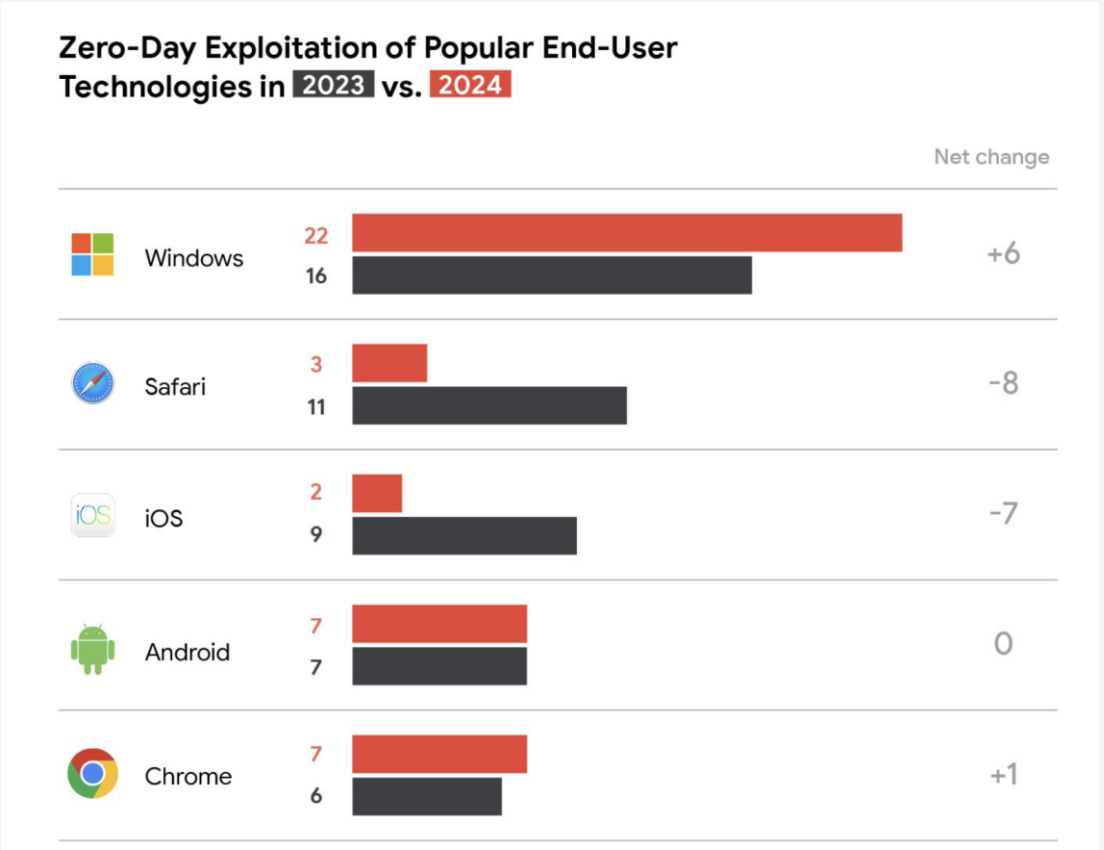
Kernel Exploitation is Getting Harder

- Windows kernel components like ntoskrnl and win32k have been key LPE targets
- Some mitigations now make kernel bugs harder to find

Google Project Zero

0day “In the Wild” (2023 ~ 2025)





[1] https://cloud.google.com/blog/topics/threat-intelligence/2024-zero-day-trends?hl=en

Exploring New Attack Vectors

- Based on the following conditions:
  1) Services running with **high privileges**
  2) Services accessible from **low-privileged** users or **AppContainers**



New Attack Vectors: COM

- We Found COM!



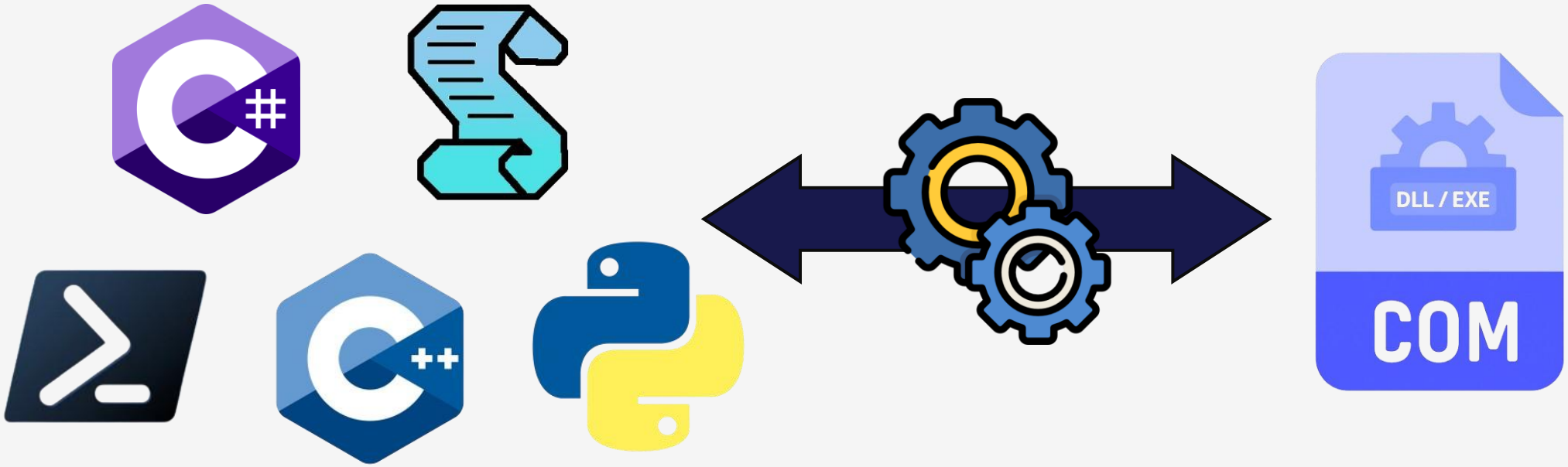
What is COM?

- Many Windows features make use of **COM**
  ex) Windows Explorer, Microsoft Office, Windows Firewall, Speech API...
- So, What is **COM**?



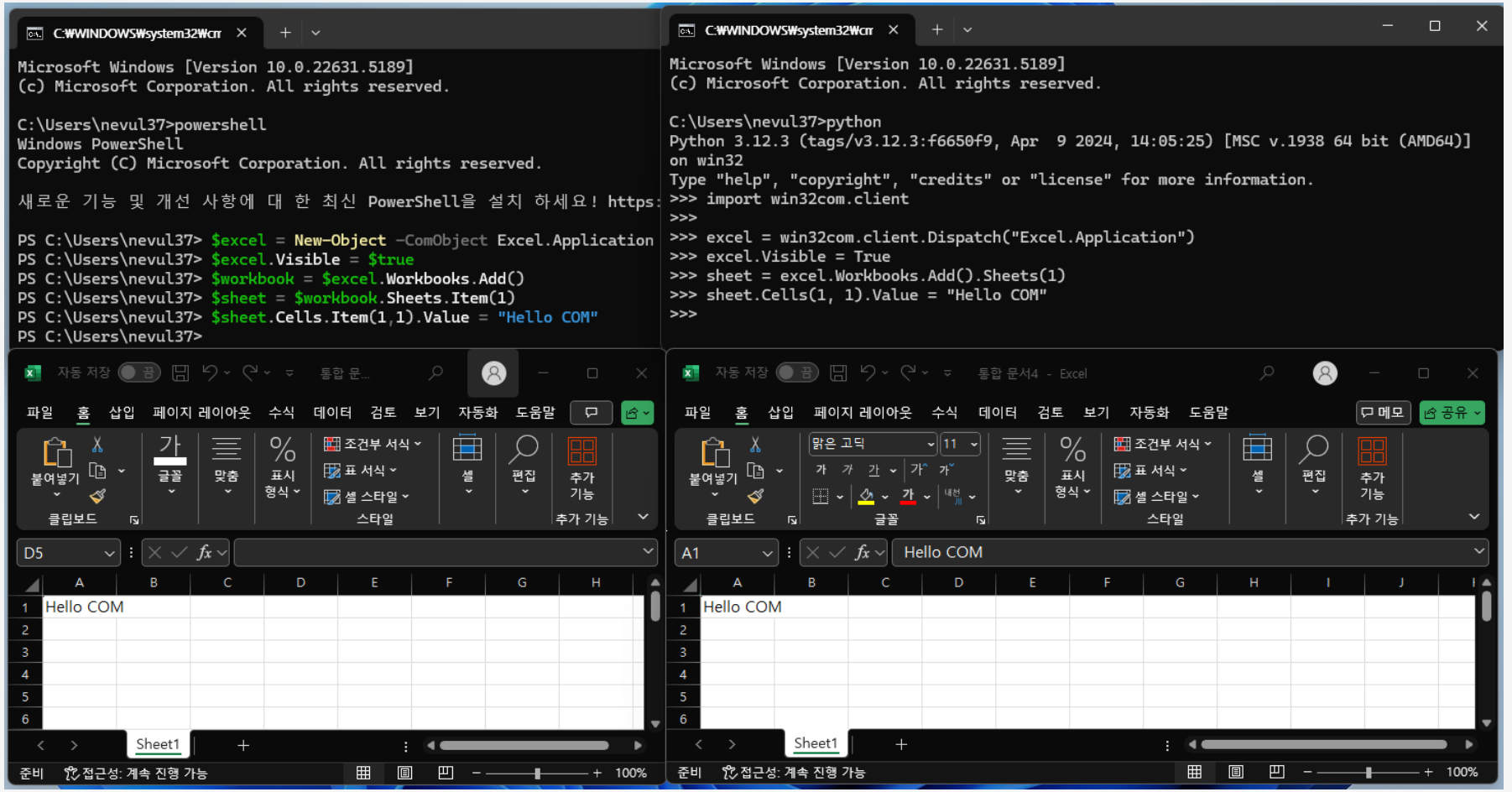
Basic Concepts of COM

- **COM** is a Microsoft-designed architecture for creating and interacting with objects across processes and modules
- **COM** supports multiple programming languages



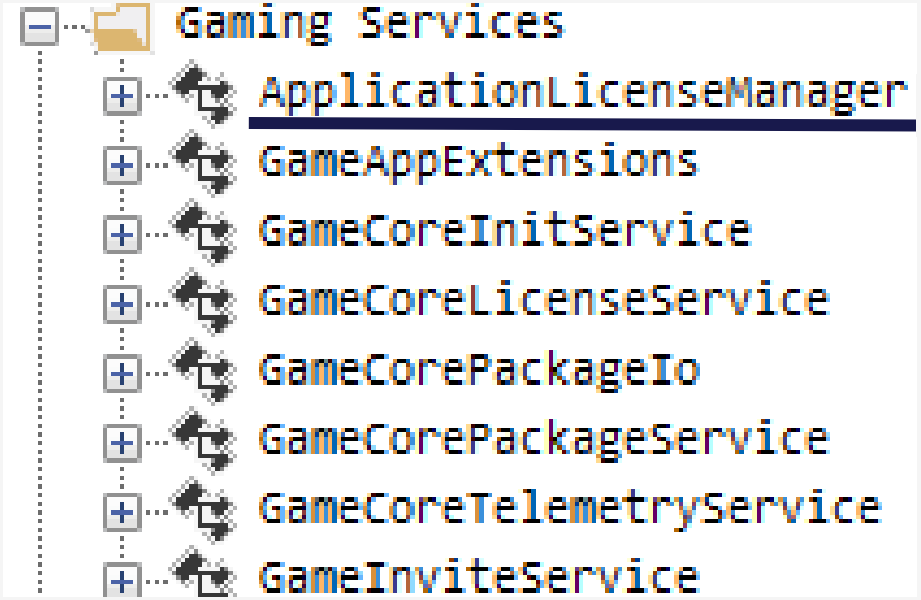
Language Independent

- **PowerShell** and **Python** can use **COM** to write data to Excel in the same way!

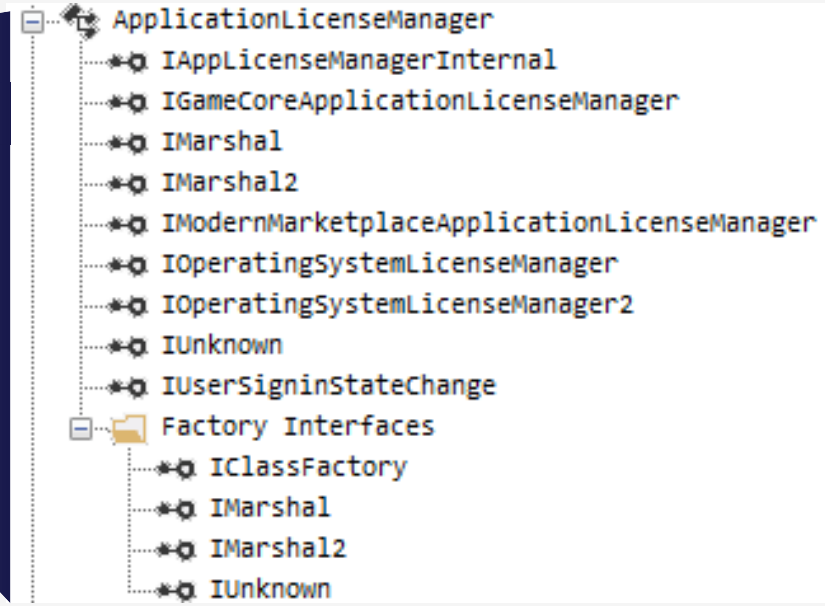


How is This Possible?

- COM identifies objects using **CLSIDs (Class IDs)** and **IIDs (Interface IDs)**
- **CLSID** defines how to create the object, **IID** defines how to use its interface.
- This allows different languages to share the same binary interface



CLSIDs

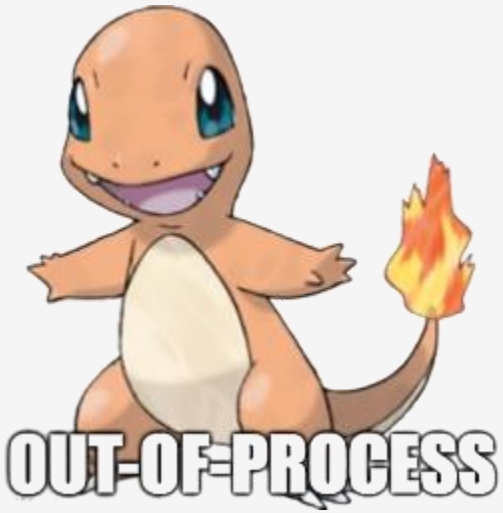


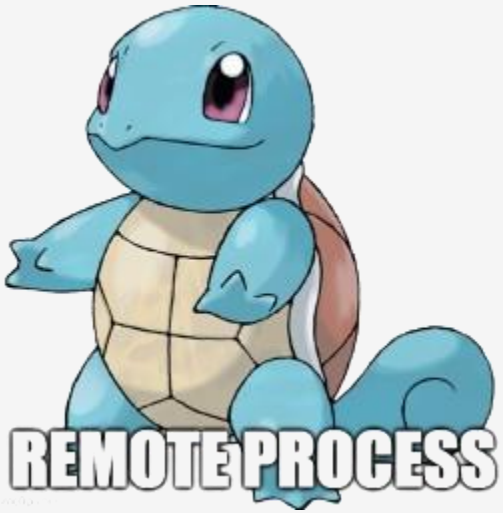
IIDs

How is This Possible?

- COM object behavior depends on its server type
- There are **three** main types of COM servers



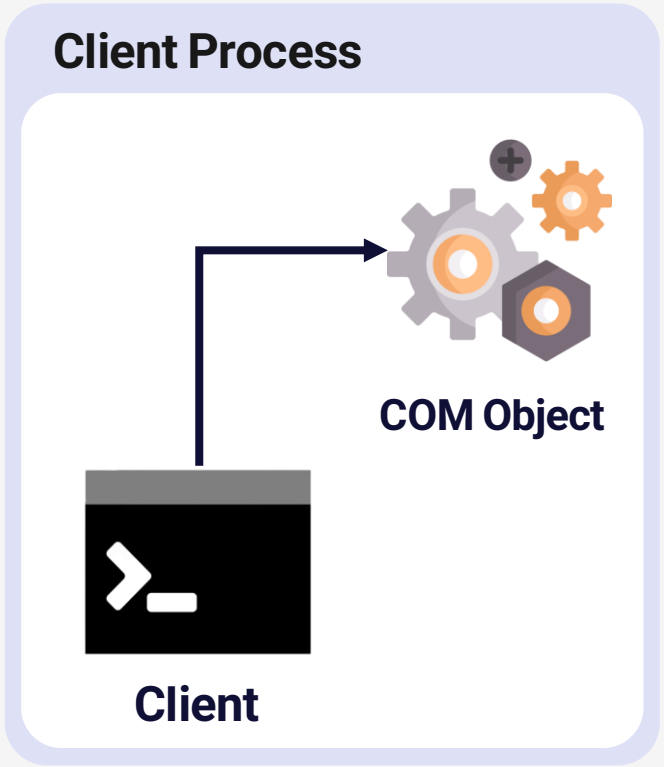




[1] https://imgflip.com

In-Process Server

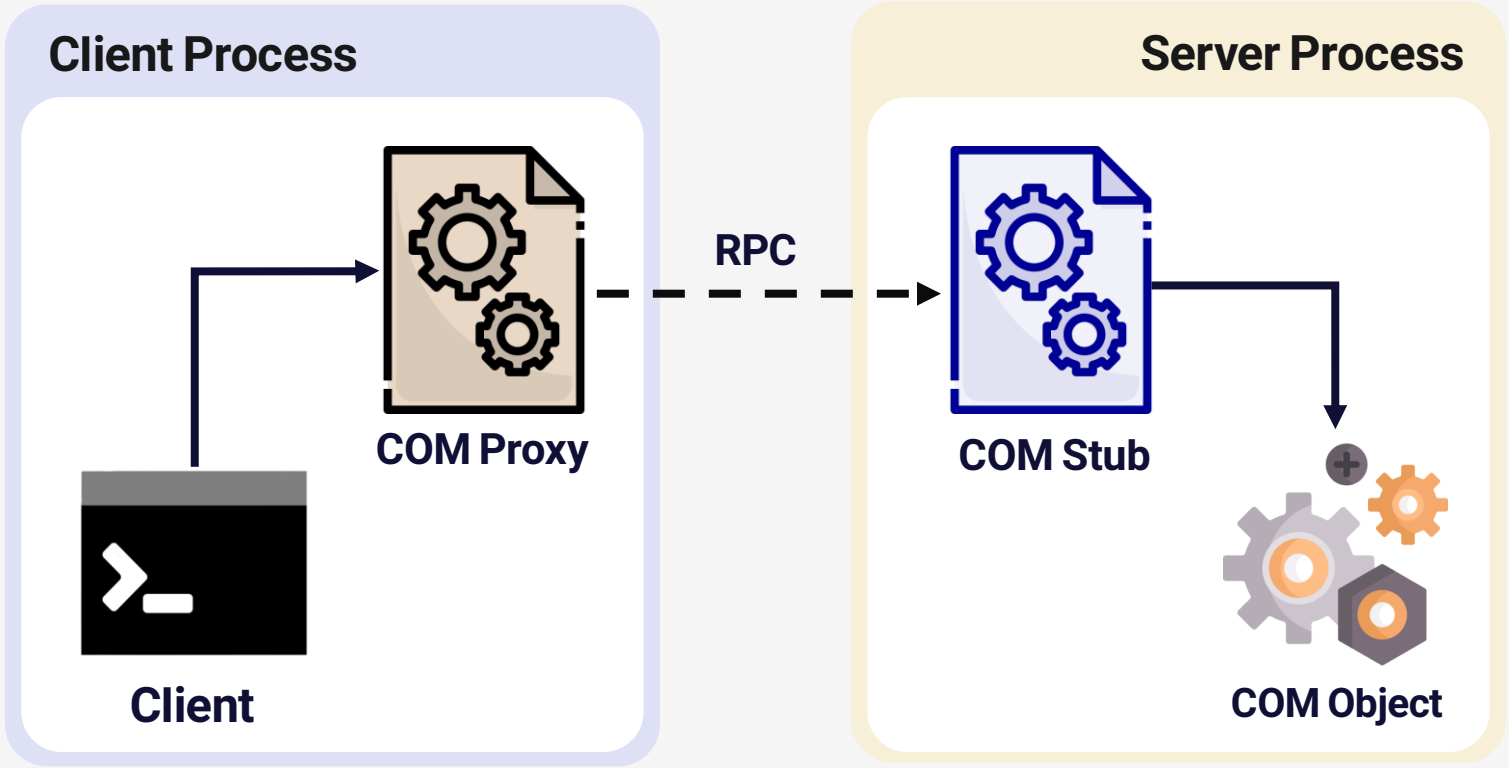
- **In-Process Server** (InProcServer) runs as a DLL inside the client process
- It shares the same privileges as the client process



[1] Design Reference: https://docwiki.embarcadero.com/RADStudio/Athens/en/In-process,_Out-of-process,_and_Remote_Servers

Out-of-Process Server

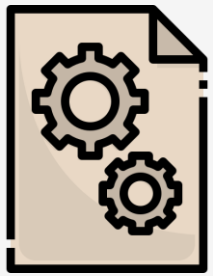
- **Out-of-Process Server** (LocalServer) runs in a separate process from the client
- Most are hosted by svchost.exe loading the COM server DLL



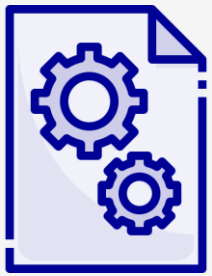
[1] Design Reference: https://docwiki.embarcadero.com/RADStudio/Athens/en/In-process,_Out-of-process,_and_Remote_Servers

COM Data Transfer

- COM enables cross-process communication using **Proxy/Stub** and **Marshalling**
- These are essential for LocalServer and Remote Server models



COM Proxy



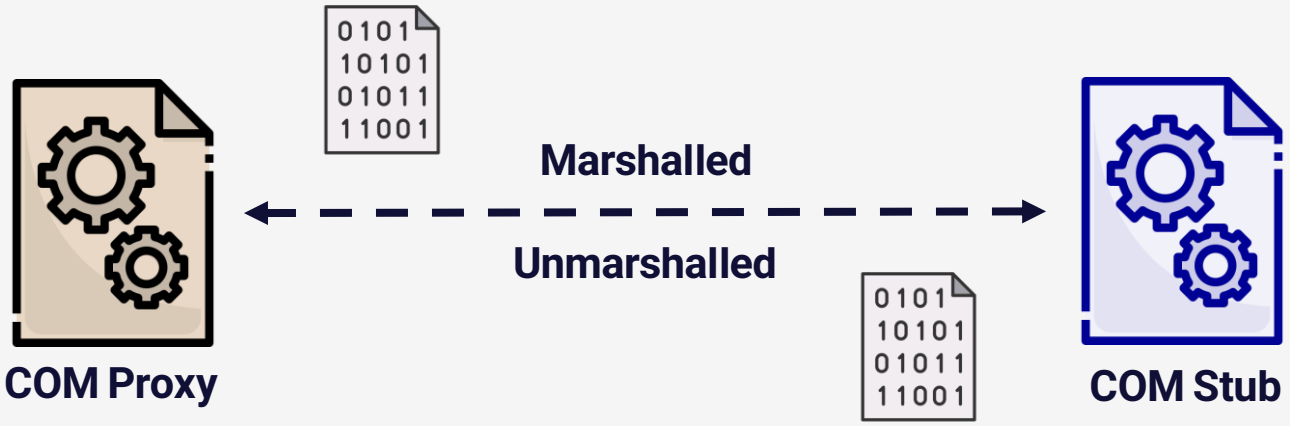
COM Stub



Marshalling

Proxy / Stub

- **Proxy** handles calls on the **client** side, **Stub** on the **server** side
- Proxy and stub transfer method call data via **Marshalling** and **Unmarshalling**



Marshalling / Unmarshalling

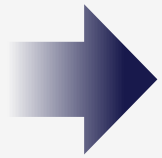
- **Marshalling** converts method calls and data into a **transferable format**
- Both Marshalling and Unmarshalling must strictly follow **IDL** definitions
  - Incorrect marshalling causes RPC errors or data corruption!



Client


```
[
  object,
  uuid(E341516B-2E32-11D1-9964-00C04FBBB345),
]
interface IEventPublisher : IDispatch {
    HRESULT Proc7([out] BSTR* p0);
    HRESULT Proc8([in] BSTR p0);
    HRESULT Proc9([out] BSTR* p0);
    HRESULT Proc10([in] BSTR p0);
    HRESULT Proc11([out] BSTR* p0);
    HRESULT Proc12([in] BSTR p0);
    HRESULT Proc13([out] BSTR* p0);
    HRESULT Proc14([in] BSTR p0);
    HRESULT Proc15([out] BSTR* p0);
    HRESULT Proc16([in] BSTR p0);
    HRESULT Proc17([in] BSTR p0, [out] VARIANT* p1);
    HRESULT Proc18([in] BSTR p0, [in] VARIANT* p1);
    HRESULT Proc19([in] BSTR p0);
    HRESULT Proc20([out] IEventObjectCollection** p0);
}
```

IDL

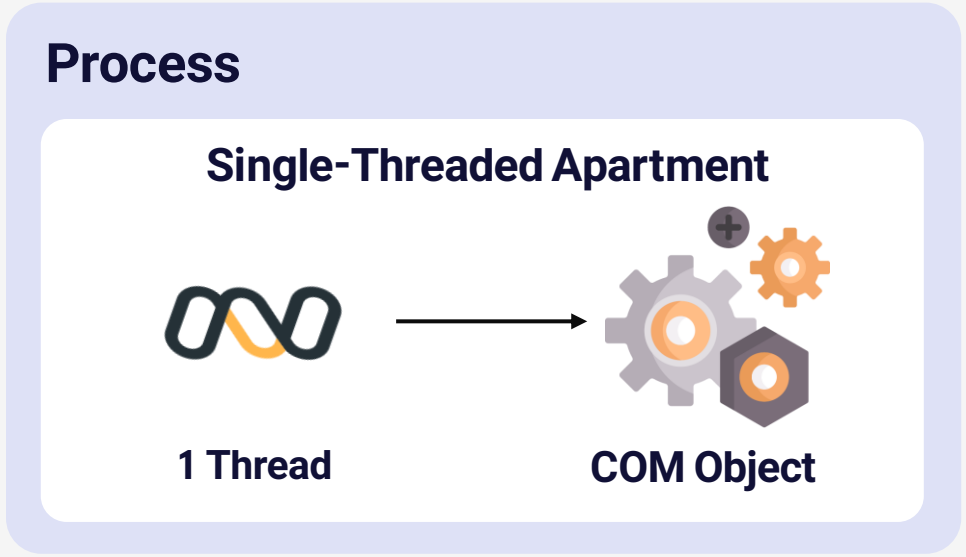




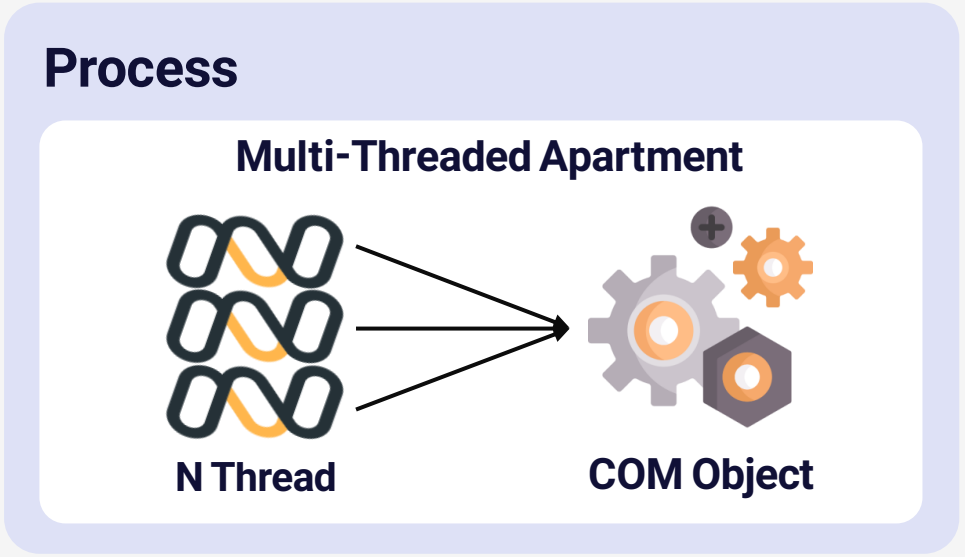
Remote COM Object

COM Threading Model

- COM uses the concept of **Apartments** to define thread execution contexts
- **Apartment** determines how a COM object is called and synchronized



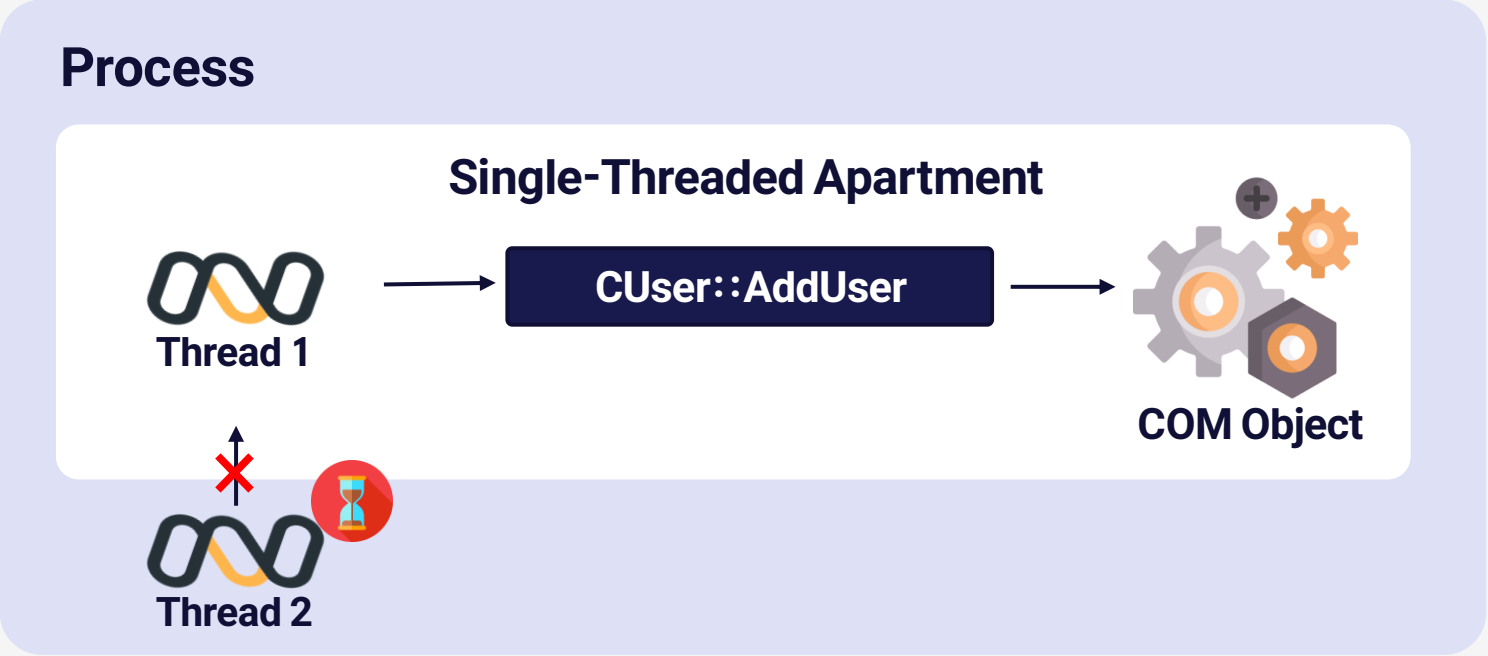
STA



MTA

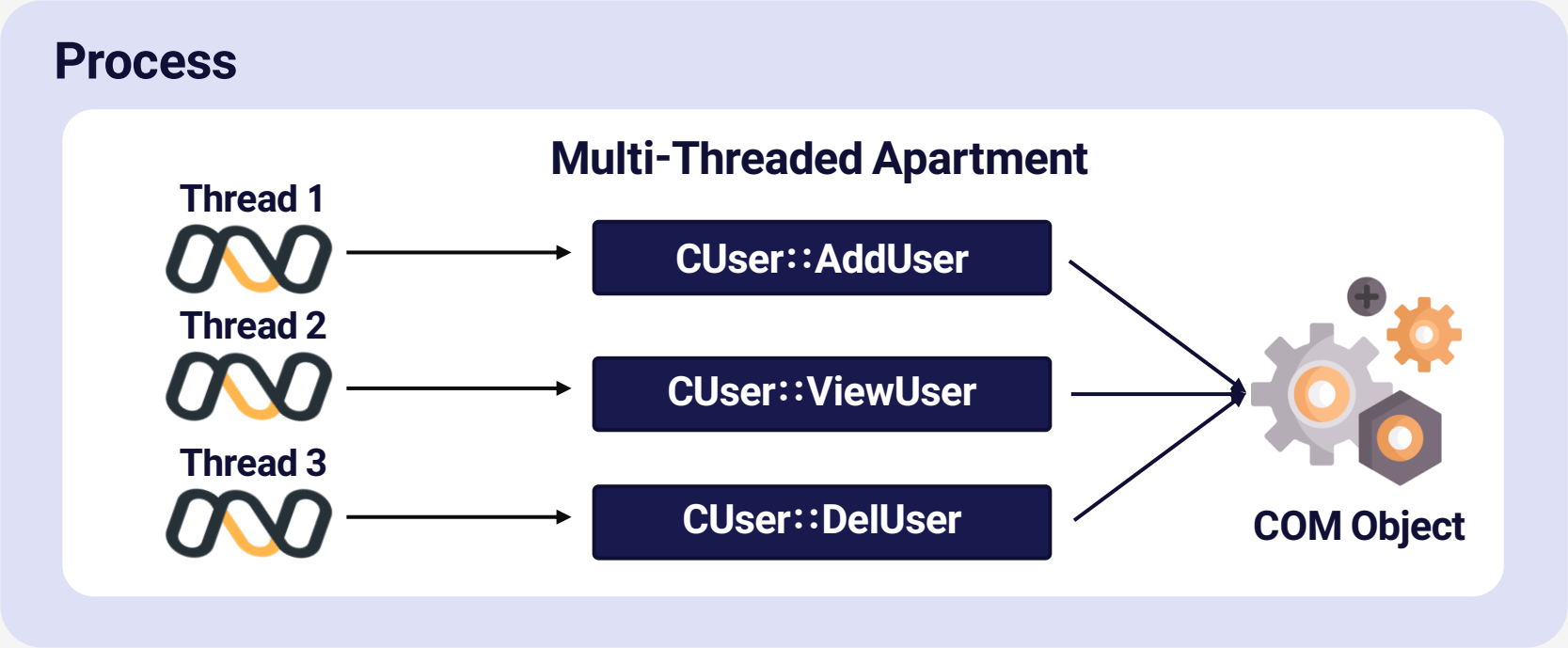
STA (Single-Threaded Apartment)

- **STA** limits each apartment to a single thread
- COM handles thread safety via a message queue, so **no manual sync is needed**



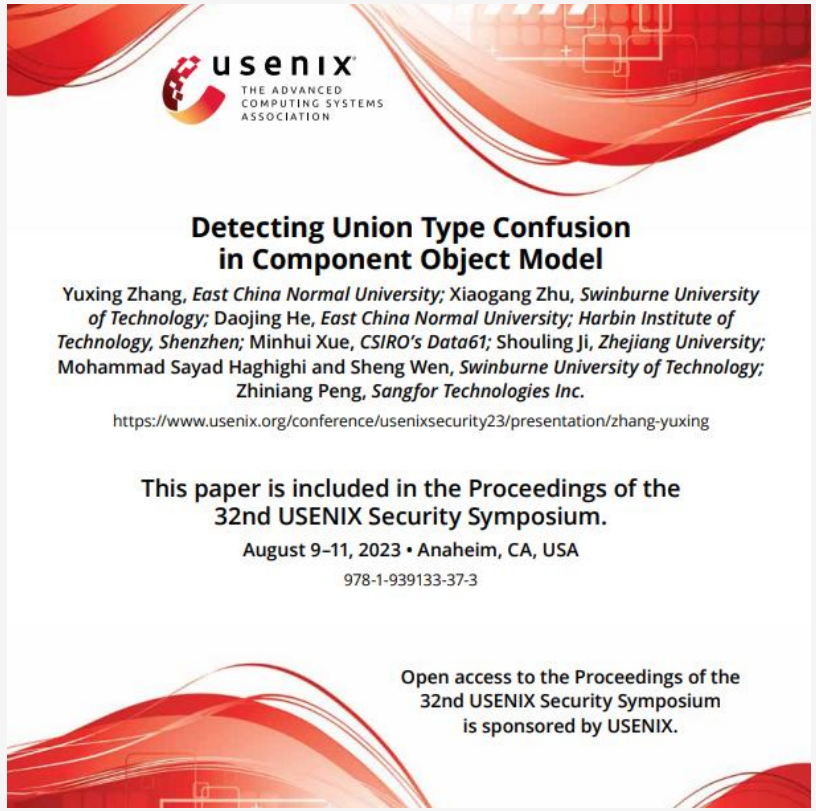
MTA (Multi-Threaded Apartment)

- **MTA** allows multiple threads in an Apartment (Yes, race conditions!)
- COM server **must implement their own** thread safety



Learning from the Past

- This was our first attempt at analyzing COM vulnerabilities...
- So before diving into COM vulnerability research, we reviewed prior works :D



usenix
THE ADVANCED COMPUTING SYSTEMS ASSOCIATION

Detecting Union Type Confusion in Component Object Model

Yuxing Zhang, *East China Normal University;* Xiaogang Zhu, *Swinburne University of Technology;* Daojing He, *East China Normal University; Harbin Institute of Technology, Shenzhen;* Minhui Xue, *CSIRO's Data61;* Shouling Ji, *Zhejiang University;* Mohammad Sayad Haghighi and Sheng Wen, *Swinburne University of Technology;* Zhiniang Peng, *Sangfor Technologies Inc.*

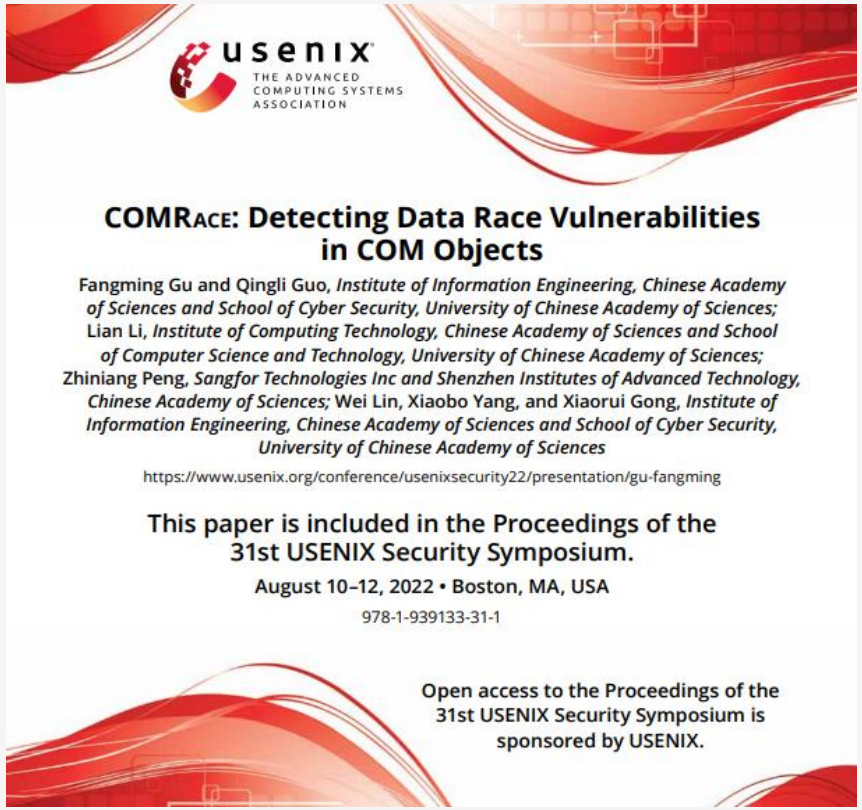
https://www.usenix.org/conference/usenixsecurity23/presentation/zhang-yuxing

This paper is included in the Proceedings of the 32nd USENIX Security Symposium.

August 9–11, 2023 • Anaheim, CA, USA

978-1-939133-37-3

Open access to the Proceedings of the 32nd USENIX Security Symposium is sponsored by USENIX.



usenix
THE ADVANCED COMPUTING SYSTEMS ASSOCIATION

COMRace: Detecting Data Race Vulnerabilities in COM Objects

Fangming Gu and Qingli Guo, *Institute of Information Engineering, Chinese Academy of Sciences and School of Cyber Security, University of Chinese Academy of Sciences;* Lian Li, *Institute of Computing Technology, Chinese Academy of Sciences and School of Computer Science and Technology, University of Chinese Academy of Sciences;* Zhiniang Peng, *Sangfor Technologies Inc and Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences;* Wei Lin, Xiaobo Yang, and Xiaorui Gong, *Institute of Information Engineering, Chinese Academy of Sciences and School of Cyber Security, University of Chinese Academy of Sciences*

https://www.usenix.org/conference/usenixsecurity22/presentation/gu-fangming

This paper is included in the Proceedings of the 31st USENIX Security Symposium.

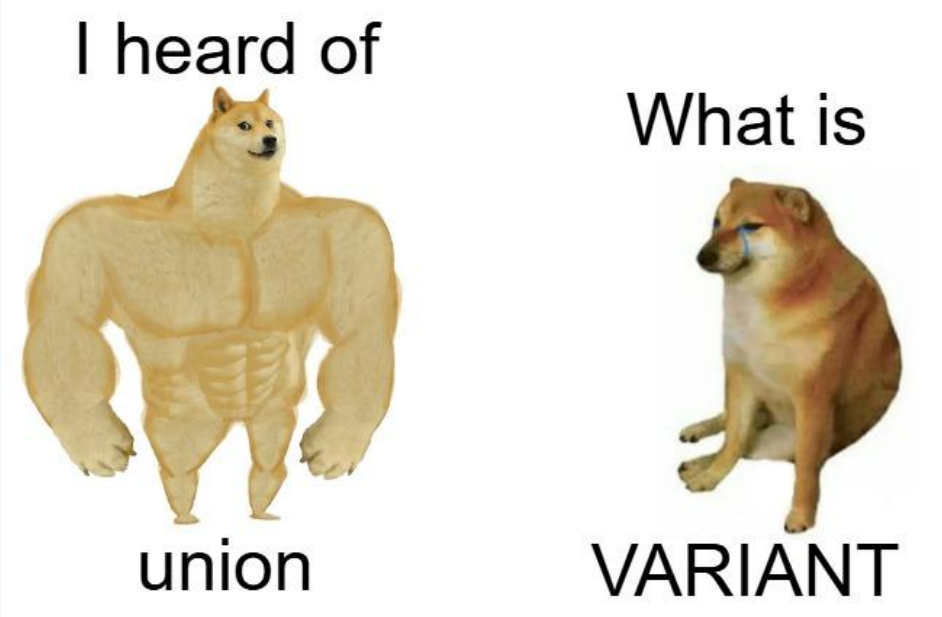
August 10–12, 2022 • Boston, MA, USA

978-1-939133-31-1

Open access to the Proceedings of the 31st USENIX Security Symposium is sponsored by USENIX.

Prior Work – COMFUSION

- **COMFUSION** shows that Union and **VARIANT** types in COM can cause **Type Confusion**
- union, I know... **VARIANT**? Not so much



[1] https://imgflip.com

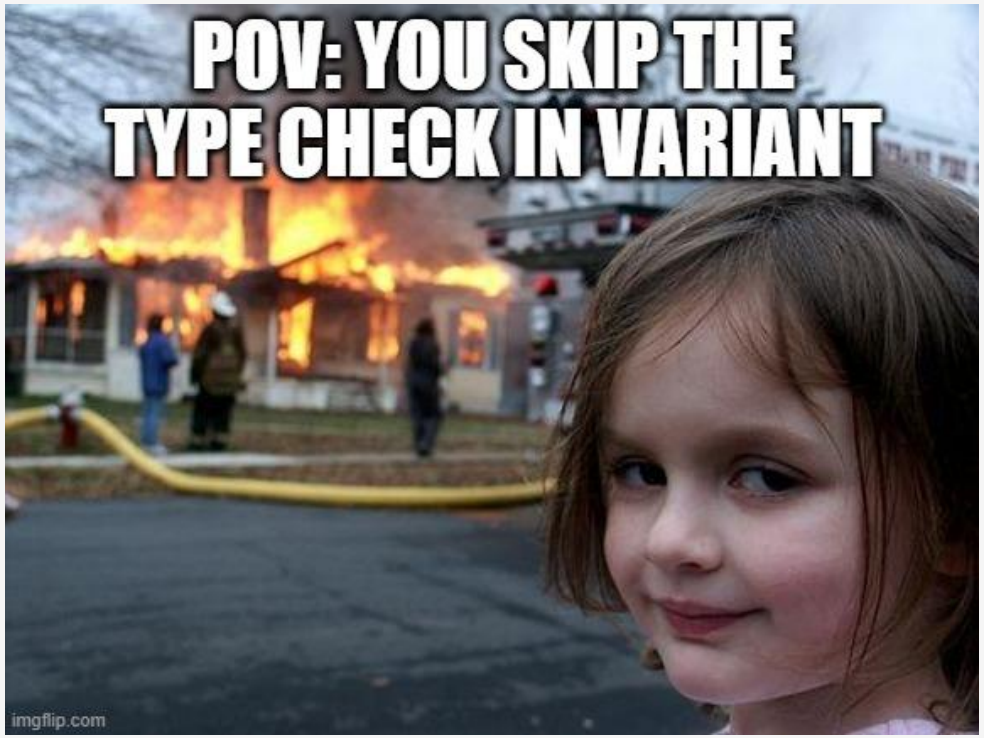
What is VARIANT?

- **VARIANT** stores both type "**vt**" and value
- Check "vt" first. Skip it? Say hello to **Type Confusion :O**

VarType	Hex	Propvariant Type	Propvariant Member	TypeName	Data bytes	C and C++ type
0	0x00	VT_EMPTY	None	Empty[1]		
1	0x01	VT_NULL	None	Null[2]		
2	0x02	VT_I2	iVal	Integer	2A00	SHORT
3	0x03	VT_I4	lVal	Long	2A000000	LONG
4	0x04	VT_R4	fltVal	Single	00002842	FLOAT
5	0x05	VT_R8	dblVal	Double	0000000000004540	DOUBLE
6	0x06	VT_CY	cyVal	Currency	A068060000000000	CY structure
7	0x07	VT_DATE	date	Date	00000000C0D5E140	DATE (double)
8	0x08	VT_BSTR	bstrVal	String	xxxxxxxx	(BSTR):(OLECHAR *):(WCHAR *):(wchar_t *)
9	0x09	VT_DISPATCH	pdispVal			
10	0x0a	VT_ERROR	scode	Error	2A000A80	HRESULT (long int)
11	0x0b	VT_BOOL	boolVal	Boolean	FFFF	VARIANT_BOOL (short)
12	0x0c	VT_VARIANT	pvarVal	Variant		VARIANT
13	0x0d	VT_UNKNOWN	punkVal	Nothing[4]	00000000	IUnknown *
14	0x0e	VT_DECIMAL	decVal			DECIMAL

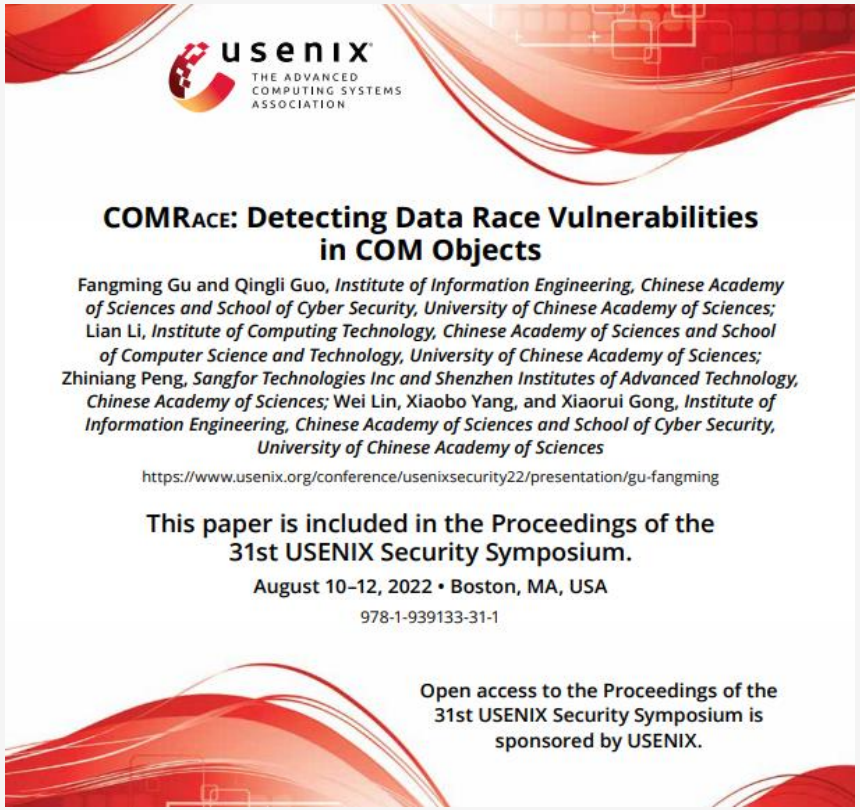
Marshalling != Security

- Marshalling relies on IDL, not on inspecting VARIANT internals
- Lack of server-side validation can lead to **Type Confusion :D**



Prior Work – COMRace

- COMRace detected Race Conditions with a specialized tool
- And then... We saw interesting pattern



usenix
THE ADVANCED COMPUTING SYSTEMS ASSOCIATION

COMRace: Detecting Data Race Vulnerabilities in COM Objects

Fangming Gu and Qingli Guo, *Institute of Information Engineering, Chinese Academy of Sciences and School of Cyber Security, University of Chinese Academy of Sciences;* Lian Li, *Institute of Computing Technology, Chinese Academy of Sciences and School of Computer Science and Technology, University of Chinese Academy of Sciences;* Zhiniang Peng, *Sangfor Technologies Inc and Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences;* Wei Lin, Xiaobo Yang, and Xiaorui Gong, *Institute of Information Engineering, Chinese Academy of Sciences and School of Cyber Security, University of Chinese Academy of Sciences*

https://www.usenix.org/conference/usenixsecurity22/presentation/gu-fangming

This paper is included in the Proceedings of the 31st USENIX Security Symposium.

August 10–12, 2022 • Boston, MA, USA

978-1-939133-31-1

Open access to the Proceedings of the 31st USENIX Security Symposium is sponsored by USENIX.

Interesting Pattern

- CVE-2020-1146: **Double Free** in MTA via put_AuthData method

```
Windows::System::Internal::SignInContext::put_data(*this, HSTRING a2) {
  Microsoft::WRL::Wrappers::HString::Set(this + 15, &a2);
}

__int64 HString::Set(HSTRING *newString, HSTRING *a2) {
  unsigned int v2;
  v2 = 0;
  if(!*a2 || *a2 != *newString) {
     WindowsDeleteString(*newString); // Delete
     *newString = 0;
     v2 = WindowsDuplicateString(*a2, newString);
  }
}
```

What We Learned and ...

- We revisited COM bugs around Type Confusion and Race Conditions
- Most bugs were from **2020**, which raised the question — what might have been overlooked?
- We began our own journey, focusing on these **two keywords**



Discovered Vulnerabilties

- Spoiler: 10+ bugs, and CVEs
- We'll introduce **4 vulnerabilities** we found particularly interesting

4 Bugs :D

- Case 87975: **Type Confusion** in LxpSvc
- CVE-2025-27475: **Double Free Bug** in InstallService
- CVE-2024-49095: **Use-After-Free** in PrintWorkflowUserSvc
- CVE-2025-21234: **Improper Input Validation** in PrintWorkflowUserSvc



Case 87975: Type Confusion in LxpSvc

```
__int64 __fastcall DeviceLanguageManager::SetLanguageOperationState(
        DeviceLanguageManager *this,
        const unsigned __int16 *a2,
        unsigned int a3,
        struct tagVARIANT *a4)
{
  __int64 v5; // rax
  int v8; // ebx
  __int64 v9; // rdx
  IRecordInfo *pRecInfo; // xmm1_8
  __int64 v12[2]; // [rsp+20h] [rbp-28h] BYREF
  IRecordInfo *v13; // [rsp+30h] [rbp-18h]

  *(_QWORD *)&v12 = a2;
  v5 = -1LL;
  do
    ++v5;
  while ( a2[v5] );
  *((_QWORD *)&v12 + 1) = v5;
  if ( !(unsigned __int8)bcp47::IsValid(&v12) )
  {
  }
  v12[0] = *(_OWORD *)&a4->vt; // [1] Get type for VARIANT
  v12[1] = a4->pRecInfo;  // [2] Get data from VARIANT
  v13 = pRecInfo;
  v8 = SetLanguageOperationStateInRegistry(a2, a3, &v12);
}
```

Case 87975: Type Confusion in LxpSvc

```
__int64 __fastcall SetLanguageOperationStateInRegistry(
  __int64 a1, int a2, struct tagVARIANT *a3)
{
 CreateLanguageOverlayKey(&v14, 0LL, L"Operations");
 CreateRegKey(&hKey, v14, a1);
 v15 = 0;
 if ( a2 ) // [1] check a2
 {
   v8 = a2 - 1;
   if ( v8 )
   {
     v9 = v8 - 1;
     ...
     ...
     data = a3->iVal == -1;
     goto LABEL_17;
   }
   v11 = L"ErrorCode";
 }
 else
 {
   v11 = L"Status";
 }
 data = a3->lVal; // [2] If a2 is 0, 'data' comes from user-supplied VARIANT
LABEL_17:
 SetRegValue(hKey, v6, v11, 4u, 4u, &data);
}
```

```
void __fastcall SetRegValue(
  HKEY a1,
  const unsigned __int16 *a2,
  const unsigned __int16 *a3,
  DWORD a4,
  DWORD cbData,
  LPCVOID data)
{
  unsigned int v6; // eax
  v6 = RegSetKeyValueW(a1, 0LL, a3, a4, data, cbData);
}
```

Case 87975: Type Confusion in LxpSvc

```
C++                                                    Copy

LSTATUS RegSetKeyValueW(
  [in]           HKEY    hKey,
  [in, optional] LPCWSTR lpSubKey,
  [in, optional] LPCWSTR lpValueName,
  [in]           DWORD   dwType,
  [in, optional] LPCVOID lpData,
  [in]           DWORD   cbData
);
```

`[in, optional] lpData`

The data to be stored with the specified value name.

For string-based types, such as REG_SZ, the string must be null-terminated. With the REG_MULTI_SZ data type, the string must be terminated with two null characters.

`[in] cbData`

The size of the information pointed to by the *lpData* parameter, in bytes. If the data is of type REG_SZ, REG_EXPAND_SZ, or REG_MULTI_SZ, *cbData* must include the size of the terminating null character or characters.

Case 87975: Type Confusion in LxpSvc

```
__int64 __fastcall SetLanguageOperationStateInRegistry(
  __int64 a1, int a2, struct tagVARIANT *a3)
{
 CreateLanguageOverlayKey(&v14, 0LL, L"Operations");
 CreateRegKey(&hKey, v14, a1);
 v15 = 0;
 if ( a2 ) // [1] check a2
 {
   v8 = a2 - 1;
   if ( v8 )
   {
     v9 = v8 - 1;
     ...
     ...
     data = a3->iVal == -1;
     goto LABEL_17;
   }
   v11 = L"ErrorCode";
 }
 else
 {
   v11 = L"Status";
 }
 data = a3->lVal; // [2] If a2 is 0, 'data' comes from user-supplied VARIANT
LABEL_17:
 SetRegValue(hKey, v6, v11, 4u, 4u, &data);
}
```

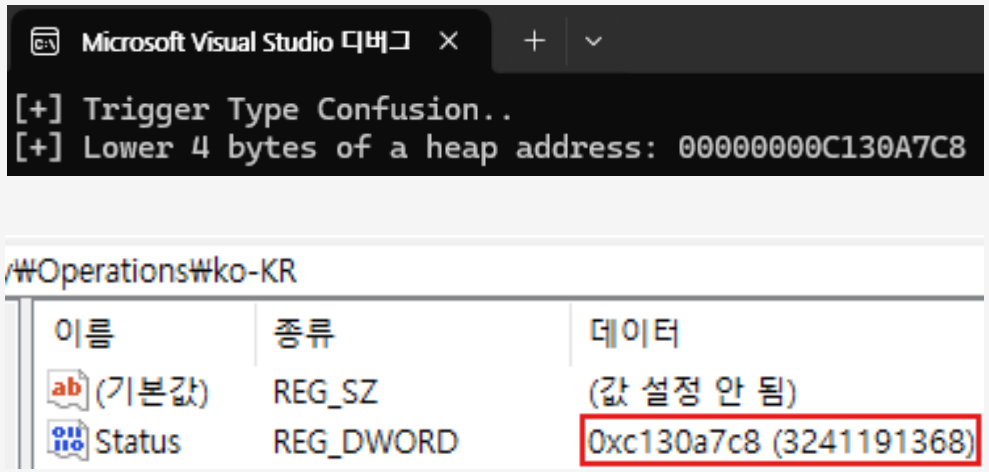
Case 87975: Type Confusion in LxpSvc

```
__int64 __fastcall SetLanguageOperationStateInRegistry(
  __int64 a1, int a2, struct tagVARIANT *a3)
{
 CreateLanguageOverlayKey(&v14, 0LL, L"Operations");
 CreateRegKey(&hKey, v14, a1);
 v15 = 0;
 if ( a2 ) //
 {
   v8 = a2 - 
   if ( v8 )
   {
     v9 = v8
     ...
     ...
     data = a
     goto LAB
   }
   v11 = L"Er
 }
 else
 {
   v11 = L"Status";
 }
 data = a3->lVal; // [2] If a2 is 0, 'data' comes from user-supplied VARIANT
LABEL_17:
 SetRegValue(hKey, v6, v11, 4u, 4u, &data);
}
```



Case 87975: Type Confusion in LxpSvc

- We can Leak Heap Address lower 4 btye
- However MSRC responded with **Won't Fix..**





CVE-2025-27475: Double Free Bug in InstallService

- Double Free Bug via Race Condition
- We need to obtain **IFulfillmentDataInfo** Object

```
__int64 __fastcall WindowsUpdate::Internal::InstallControl::CreateFulfillmentData(
        WindowsUpdate::Internal::InstallControl *this,
        HSTRING a2,
        struct WindowsUpdate::Internal::IFulfillmentDataInfo **a3)
{
  HSTRING v5; // [rsp+38h] [rbp+10h] BYREF

  v5 = a2;
  if ( Utils::IsValidProductId(a2, a2) )   // [1] Verify that a2 is 0xC bytes
    return
  MakeAndInitialize<HSTRING__ *>(a3, &v5); // [2] Return object
  *a3 = 0LL;
  return 2147942487LL;
}
```

CVE-2025-27475: Double Free Bug in InstallService

- Store HSTRING in v3 variable and Free v3
- Same Pattern in **COMRace**!!!

```
__int64 __fastcall WindowsUpdate::Internal::FulfillmentDataInfo::put_CrossGenSetId(
        WindowsUpdate::Internal::FulfillmentDataInfo *this,
        HSTRING a2)
{
  unsigned int v2; // ebx
  HSTRING *v3; // rdi

  v2 = 0;
  v3 = (HSTRING *)((char *)this + 200); // [1] Get HSTRING from this + 200
  if ( !a2 || a2 != *v3 )
  {
    WindowsDeleteString(*v3); // [2] Free HSTRING
    *v3 = 0LL;
    return (unsigned int)WindowsDuplicateString(a2, v3);
  }
  return v2;
}
```

CVE-2025-27475: Double Free Bug in InstallService

- Store HSTRING in v3 variable and Free v3
- Same Pattern in **COMRace**!!!

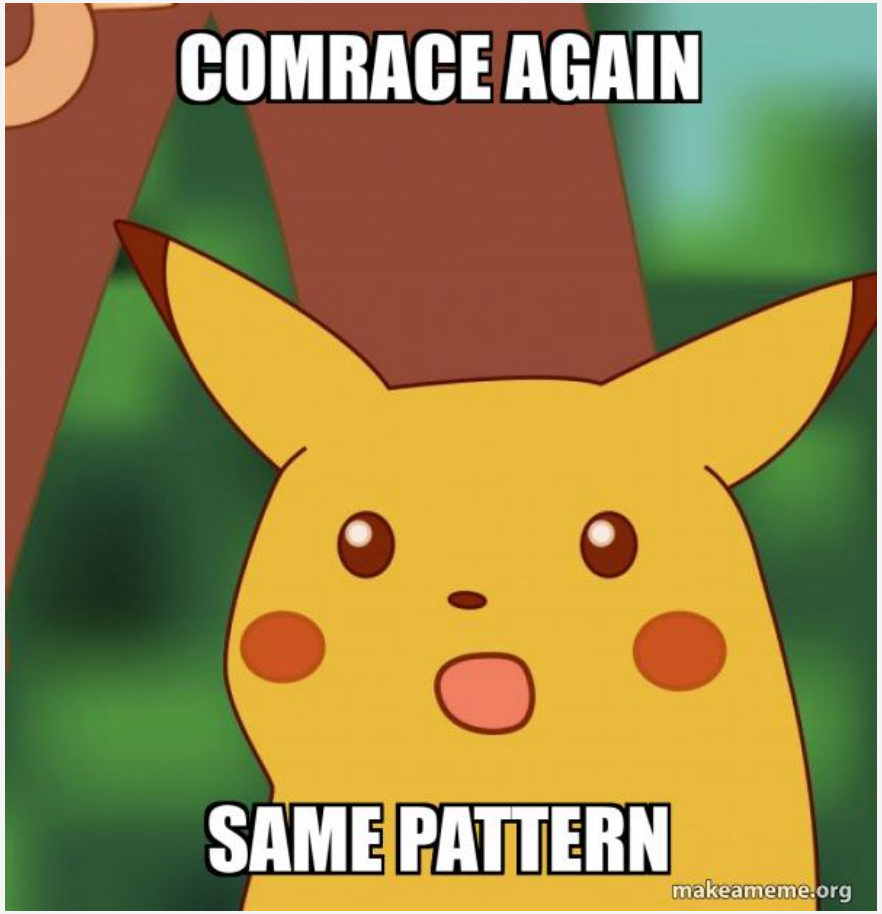
```
__int64 __fastcall WindowsUpdate::Internal::FulfillmentDataInfo::put_CrossGenSetId(
        WindowsUpdate::Internal::FulfillmentDataInfo *this,
        HSTRING a2)
{
  unsigned int v2; //
  HSTRING *v3; // rdi

  v2 = 0;
  v3 = (HSTRING *)((ch
  if ( !a2 || a2 != *v
  {
    WindowsDeleteStrin
    *v3 = 0LL;
    return (unsigned i
  }
  return v2;
}
```

```
Windows::System::Internal::SignInContext::put_data(*this, HSTRING a2) {
  Microsoft::WRL::Wrappers::HString::Set(this + 15, &a2);
}

__int64 HString::Set(HSTRING *newString, HSTRING *a2) {
  unsigned int v2;
  v2 = 0;
  if(!*a2 || *a2 != *newString) {
     WindowsDeleteString(*newString); // Delete
     *newString = 0;
     v2 = WindowsDuplicateString(*a2, newString);
  }
}
```

CVE-2025-27475: Double Free Bug in InstallService

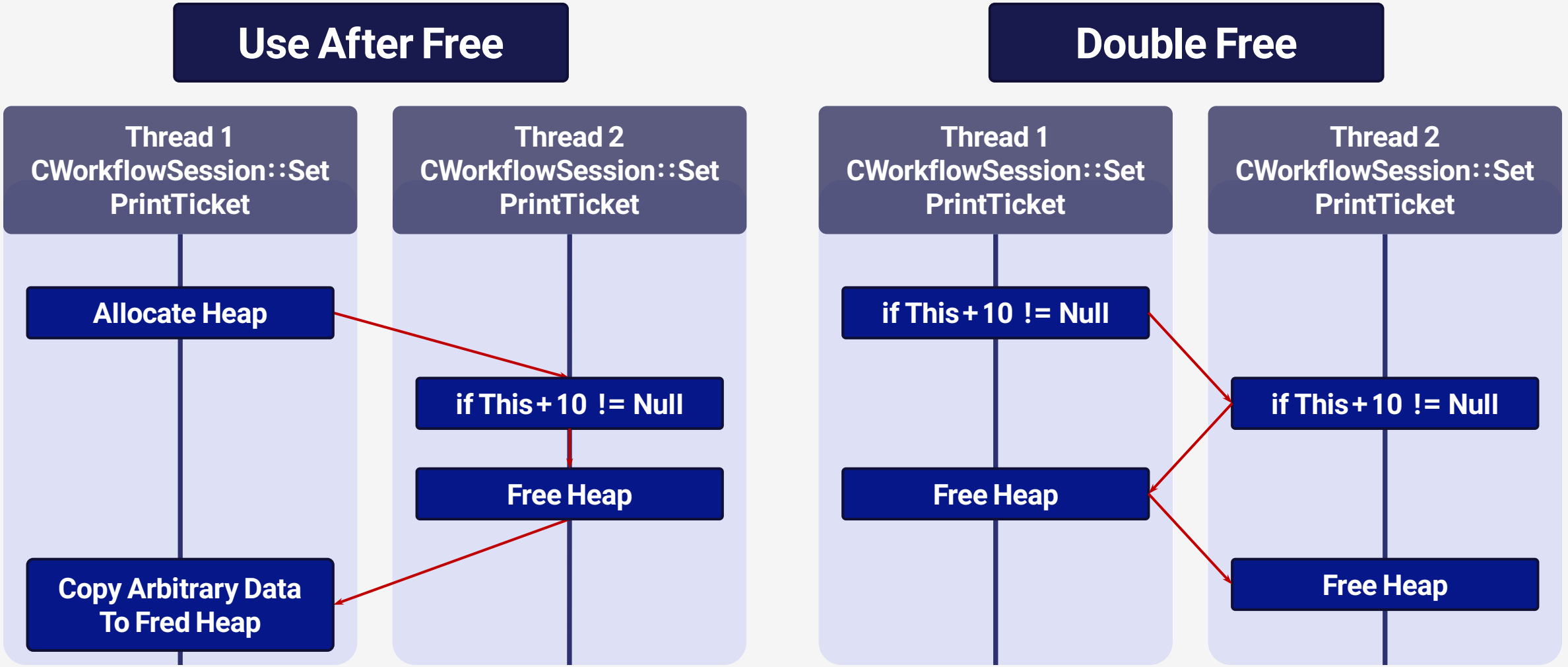


CVE-2024-49095: Use-After-Free in PrintWorkflowUserSvc

- **Use-After-free** via Race Condition

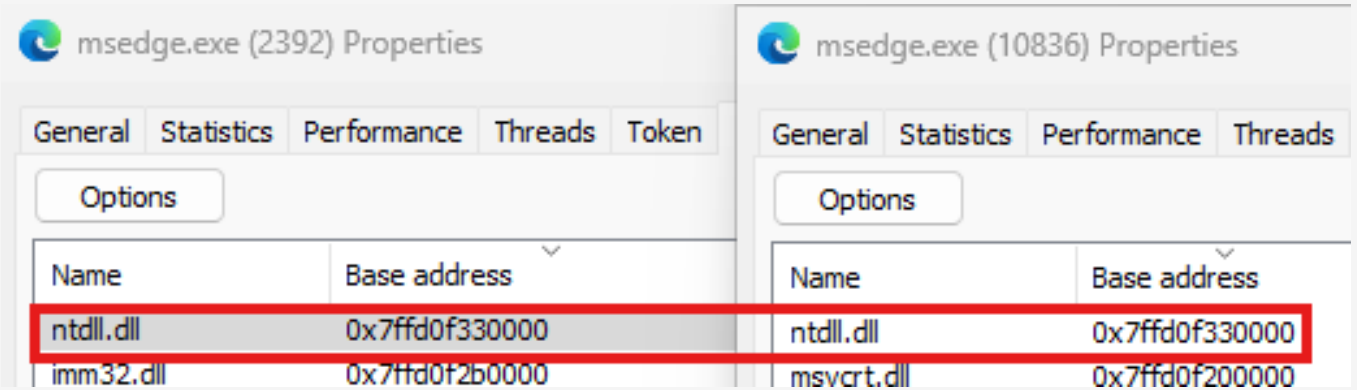
```
__int64 __fastcall CWorkflowSession::SetPrintTicket(CWorkflowSession *this, int a2, const
unsigned __int8 *a3)
{
  void *v7; // rax
  ...
  v4 = (void *)*((_QWORD *)this + 10);
  if ( v4 )
    operator delete[](v4); // [1] Free heap if already allocated
  v7 = operator new[](*((unsigned int *)this + 22));
  *((_QWORD *)this + 10) = v7;
  if ( v7 )
    memcpy_0(v7, a3, *((unsigned int *)this + 22)); // [2] Copy Data to Allocated Heap
  ...
  return v5;
}
```

CVE-2024-49095: Use-After-Free in PrintWorkflowUserSvc



CVE-2024-49095: Exploit Idea

- Windows maps the **same DLL** to the **same address** in every process
- Basic idea for Exploit IPC



CVE-2024-49095: Exploit Idea

- Heap Address Leak
- Find to other useful method that access **this+10**

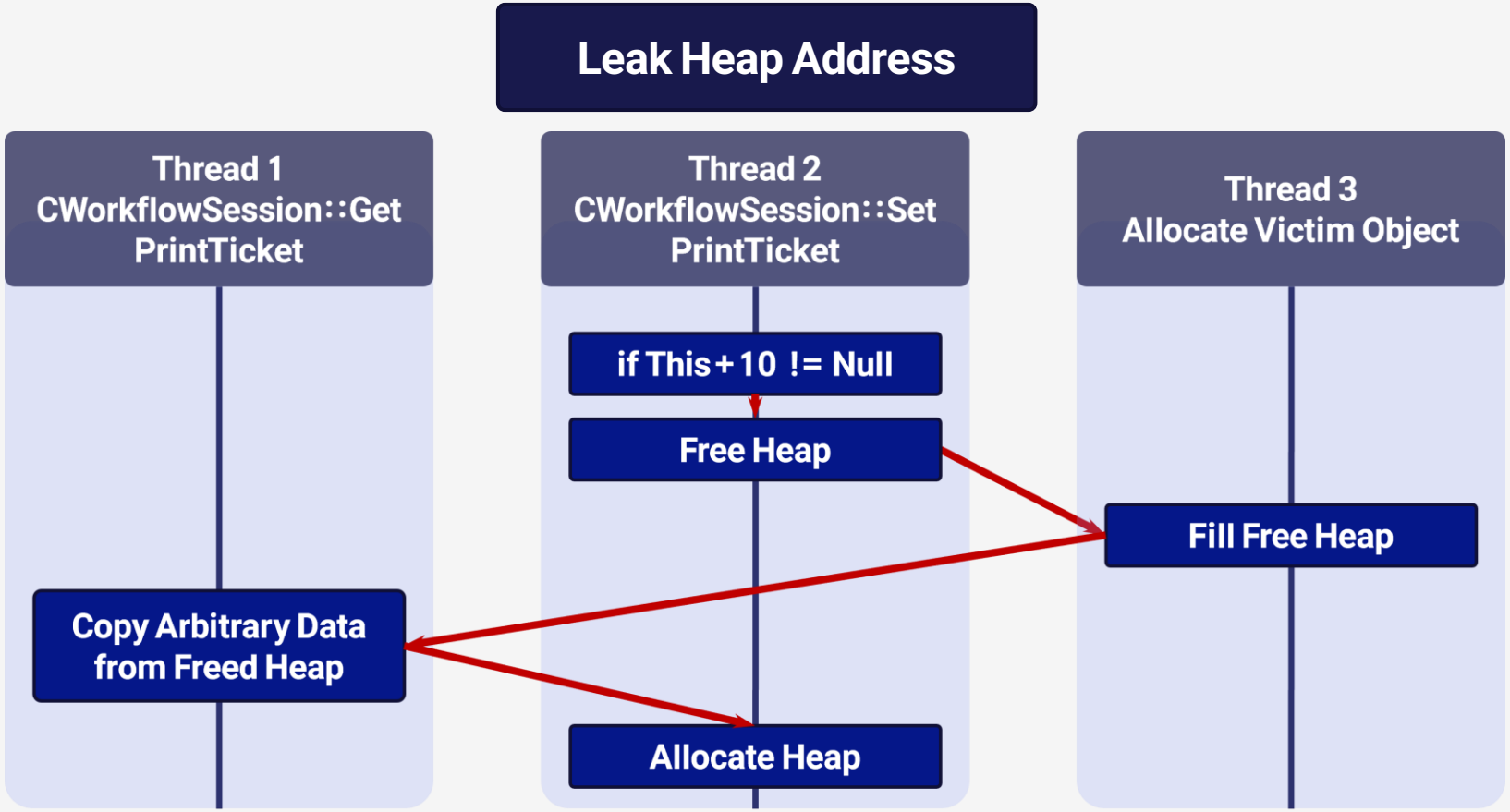
```
__int64 __fastcall CWorkflowSession::GetPrintTicket(
        CWorkflowSession *this,
        DWORD a2,
        unsigned int *a3,
        unsigned __int8 **a4)
{
  ...
    v16 = (unsigned __int8 *)CoTaskMemAlloc(*((unsigned int *)this + 22));
    //[1] allocate output buffer

    *a4 = v16;
    if ( v16 )
    {
      memcpy_0(v16, *((void **)this + 10), *((unsigned int *)this + 22));
      // [2] Copy this + 10 to output buffer

      *a3 = *((_DWORD *)this + 22);
    }
...
}
```

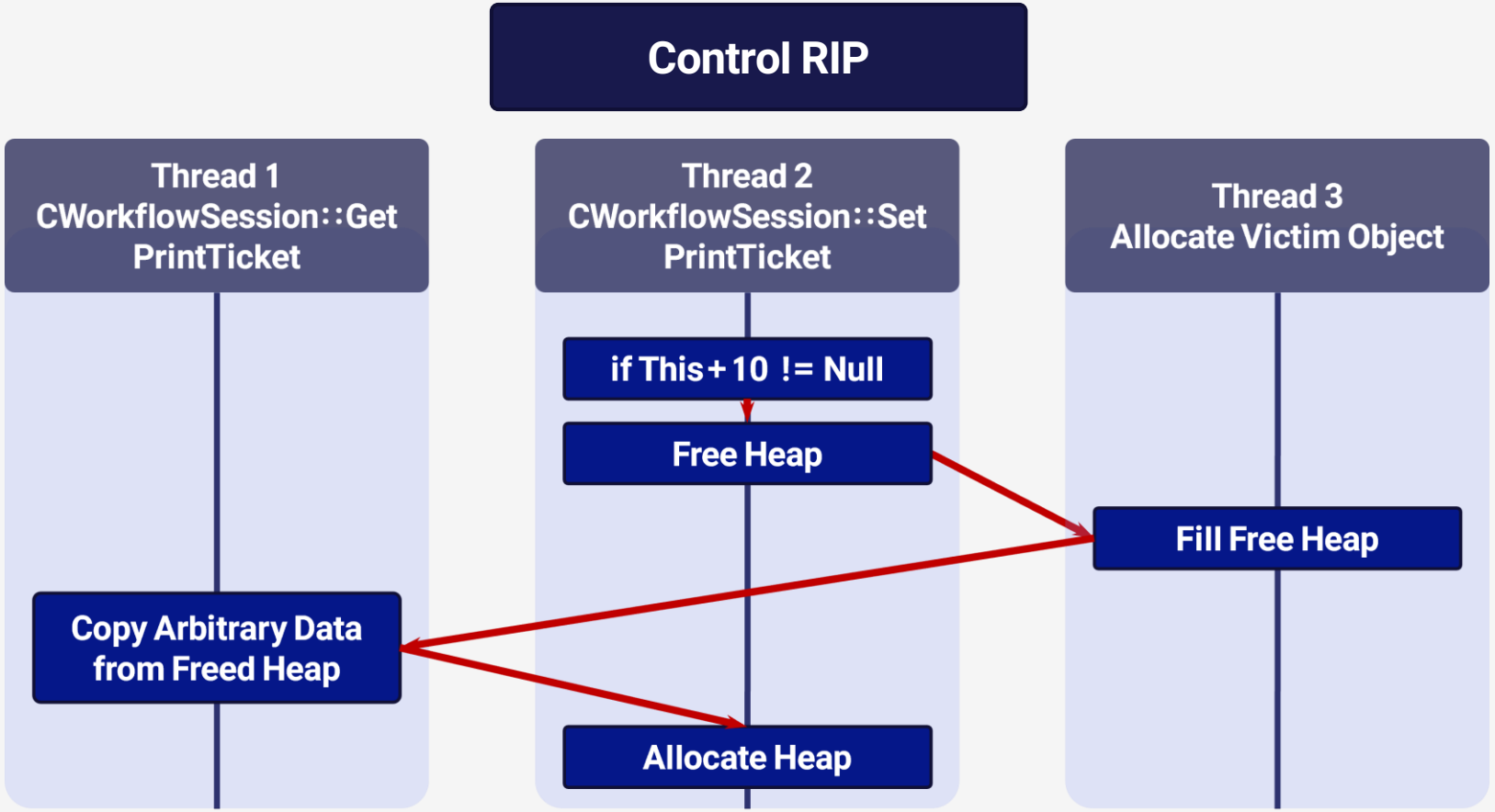
CVE-2024-49095: Exploit Idea

- Racing 3 thread
- Size and Data is Fully Controlled



CVE-2024-49095: Exploit Idea

- Racing 3 thread
- Corrupt vftable to hijack control flow



CVE-2024-49095: Why We Failed

- Double free was hard to avoid, even after a successful overwrite
- Both threads must call the same interface, and the **race window** is **extremely narrow**



CVE-2025-21234: Improper Input Validation in PrintWorkflowUserSvc

- It takes **six output-only parameters** — no inputs at all
- Would you normally analyze such a function? We wouldn't... until we found this bug :O

```
[
  object,
  uuid(B5A8BE57-53B6-4597-8D5D-4E05501F687F),
]
interface Windows.Graphics.Internal.Printing.Workflow.IWorkflowSourceSession : IInspectable {
    HRESULT Proc6();
    HRESULT Proc7([out] int* p0);
    HRESULT Proc8([in] struct Struct_1098* p0);
    HRESULT Proc9([in] int p0);
    HRESULT Proc10([in] int p0, [in, size_is(p0)]  byte[]* p1);
    HRESULT Proc11([in] HSTRING p0);
    HRESULT Proc12([out] HSTRING* p0);
    HRESULT Proc13([out] int* p0);
    HRESULT Proc14([out] int* p0, [out] __int64* p1, [out] __int64* p2, [out] byte* p3, [out] __int64* p4, [out] __int64* p5);
    HRESULT Proc15([in] int p0, [in] int p1);
    HRESULT Proc16();
}
```

CVE-2025-21234: Improper Input Validation in PrintWorkflowUserSvc

```
__int64 __fastcall IPrintSupportSourceSession::RequestSpoolingHandlesForWrite(
        __int64 a1,
        unsigned int *a2,
        unsigned __int64 *a3,
        unsigned __int64 *a4,
        unsigned __int8 *a5,
        unsigned __int64 *a6,
        unsigned __int64 *a7)
{
  __int64 v7; // rax
  __int64 v8; // rax
  __int64 v9; // rcx

  v7 = a1 + 8;
  if ( !a1 )
    v7 = 24i64;
  v8 = *(_QWORD *)v7;
  v9 = v8 + 144; // [1] v9 holds a pointer to a vftable of WorkflowSessionCommon
  if ( !v8 )
    v9 = 160i64;
  WorkflowSessionCommon::RequestSpoolingHandlesForWrite(
    (WorkflowSessionCommon *)v9, a2, a3, a4, a5, a6, a7
  ); // [2]
}
```

CVE-2025-21234: Improper Input Validation in PrintWorkflowUserSvc

- Call DuplicateHanle function

```
__int64 __fastcall WorkflowSessionCommon::RequestSpoolingHandlesForWrite(
        WorkflowSessionCommon *this,
        unsigned int *a2,
        unsigned __int64 *a3,
        unsigned __int64 *a4,
        unsigned __int8 *a5,
        unsigned __int64 *a6,
        unsigned __int64 *a7)
{
    ...
    ...
    if ( GetCallerProcessId(&dwProcessId) >= 0 ) // [1] Get Caller's PID
    {
      v21 = (char *)OpenProcess(0x40u, 0, dwProcessId); // [2] Get Caller's Process Handle
      TargetHandle = 0i64;

      v22 = (void *)*((_QWORD *)this + 2); // [3] Get *(this + 0x10)

      if ( !DuplicateHandle(GetCurrentProcess(), v22, v21, &TargetHandle, 0, 0, 2u) )
      // [4] Call DuplicateHandle
```

CVE-2025-21234: Improper Input Validation in PrintWorkflowUserSvc

```
v22 = (void *)*((_QWORD *)this + 2); // [3] Get *(this + 0x10)

if ( !DuplicateHandle(GetCurrentProcess(), v22, v21, &TargetHandle, 0, 0, 2u) )
```

C++

```
BOOL DuplicateHandle(
  [in]  HANDLE   hSourceProcessHandle,
  [in]  HANDLE   hSourceHandle,
  [in]  HANDLE   hTargetProcessHandle,
  [out] LPHANDLE lpTargetHandle,
  [in]  DWORD    dwDesiredAccess,
  [in]  BOOL     bInheritHandle,
  [in]  DWORD    dwOptions
);
```

`[in] hSourceProcessHandle`

A handle to the process with the handle to be duplicated.

The handle must have the PROCESS_DUP_HANDLE access right. For more information, see Process Security and Access Rights.

`[in] hSourceHandle`

The handle to be duplicated. This is an open object handle that is valid in the context of the source process. For a list of objects whose handles can be duplicated, see the following Remarks section.

`[in] hTargetProcessHandle`

A handle to the process that is to receive the duplicated handle. The handle must have the PROCESS_DUP_HANDLE access right.

This parameter is optional and can be specified as NULL if the **DUPLICATE_CLOSE_SOURCE** flag is set in *Options*.

CVE-2025-21234: Improper Input Validation in PrintWorkflowUserSvc

- We can duplicate service process handle to client

```
v22 = (void *)*((_QWORD *)this + 2); // [3] Get *(this + 0x10)

if ( !DuplicateHandle(GetCurrentProcess(), v22, v21, &TargetHandle, 0, 0, 2u) )
```

```
int Printing::PrintSupport::implementation::PrintSupportSession::PrintSupportSession(
        int a1, const struct winrt::hstring *a2)
{
  ...
  *(_QWORD *)(a1 + 0xA0) = &WorkflowSessionCommon::`vftable'; // this
  *(_QWORD *)(a1 + 0xA8) = 0i64;  // this + 0x8
  *(_QWORD *)(a1 + 0xB0) = -1i64; // this + 0x10 (v22)
  ...
}
```

CVE-2025-21234: Improper Input Validation in PrintWorkflowUserSvc

- Accessible from UWP and low-privileged AppContainers
- This means a sandboxed process can obtain a handle to PrintWorkflowUserSvc :D

Owner: NT AUTHORITY₩SELF
Group: BUILTIN₩Users
Integrity: Low (NoExecuteUp)

DACL | SACL

Flags: None

ACL Entries

Type	Account	Access	Flags
Allowed	NT AUTHORITY₩SYSTEM	Execute|ExecuteLocal|ActivateLocal	None
Allowed	NT AUTHORITY₩LOCAL SERVICE	Execute|ExecuteLocal|ActivateLocal	None
Allowed	NT AUTHORITY₩INTERACTIVE	Execute|ExecuteLocal|ActivateLocal	None
Allowed	APPLICATION PACKAGE AUTHORITY₩ALL APPLICATION PACKAGES	Execute|ExecuteLocal|ActivateLocal	None
Allowed	NAMED CAPABILITIES₩Lpac Printing	Execute|ExecuteLocal|ActivateLocal	None
Allowed	S-1-5-21-2702878673-795188819-444038987-2781	Execute|ExecuteLocal|ActivateLocal	None

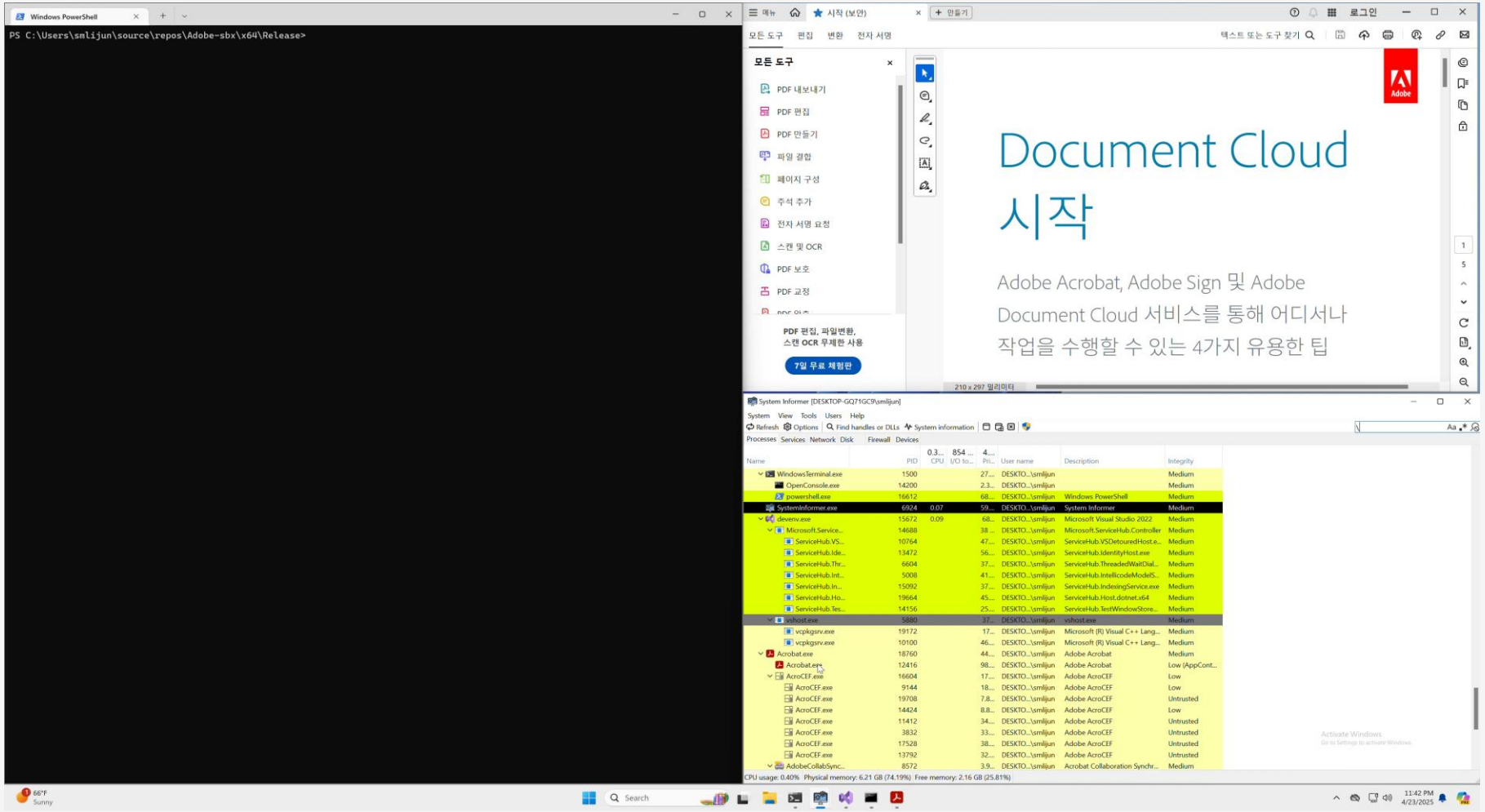
CVE-2025-21234: Improper Input Validation in PrintWorkflowUserSvc

- Memory Read/Write/Execute is now free 😊
- Classic DLL injection, Shellcode Execution, etc
- So forget those complex Race Condition bugs 🤣



Demo Time !

[Demo Link]



Official Classification: Exploit Less Likely

- Why?? 🤣

Exploitability

The following table provides an exploitability assessment for this vulnerability at the time of original publication.

Publicly disclosed
No
Exploited
No
Exploitability assessment
Exploitation Less Likely

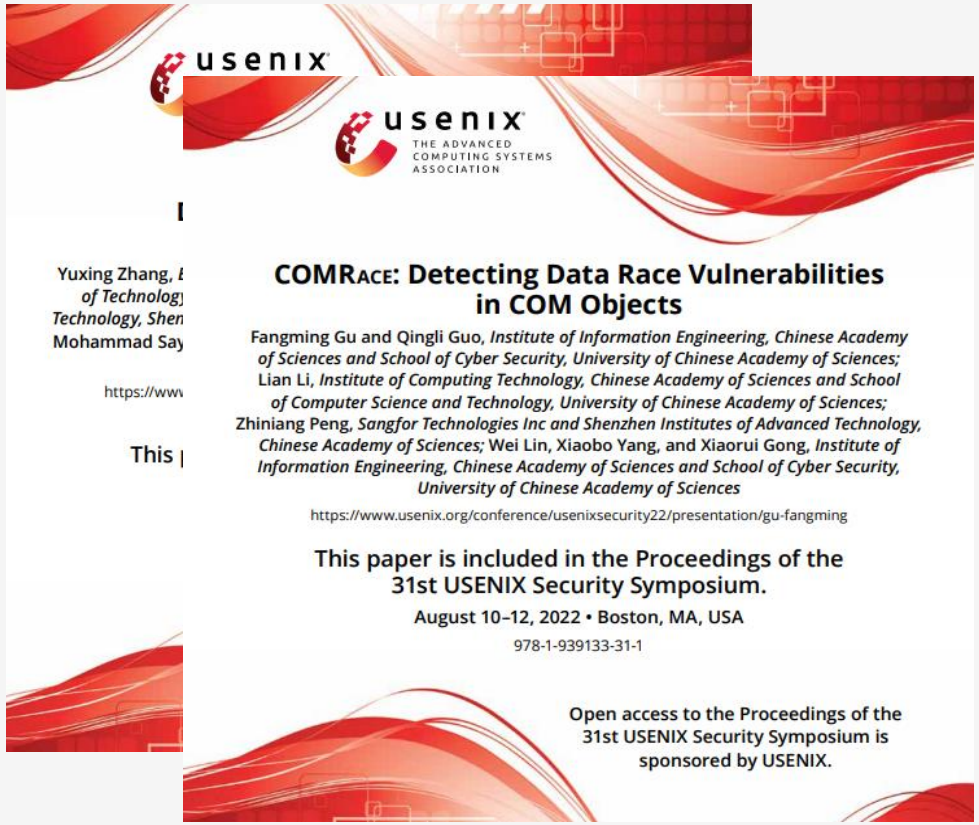
Results

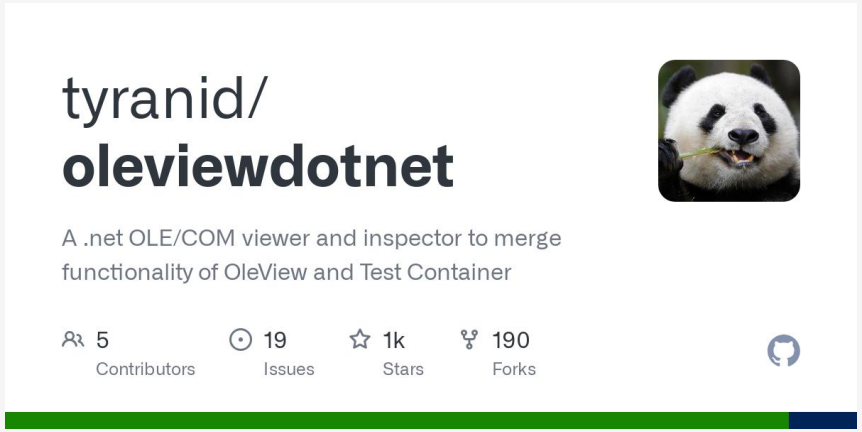
- We reported 10+ Type Confusion and Race Condition bugs to Microsoft

CVE ID	Component	Impact
CVE-2024-49095	Windows PrintWorkflowUserSvc	Elevation of Privilege & Sandbox Escape
CVE-2024-49097	Windows PrintWorkflowUserSvc	Elevation of Privilege & Sandbox Escape
CVE-2025-21234	Windows PrintWorkflowUserSvc	Elevation of Privilege & Sandbox Escape
CVE-2025-21235	Windows PrintWorkflowUserSvc	Elevation of Privilege & Sandbox Escape
CVE-2025-21182	Windows Resilient File System Deduplication Service	Elevation of Privilege
CVE-2025-21183	Windows Resilient File System Deduplication Service	Elevation of Privilege
CVE-2025-27467	Windows Digital Media	Elevation of Privilege
CVE-2025-27476	Windows Digital Media	Elevation of Privilege
CVE-2025-27730	Windows Digital Media	Elevation of Privilege
CVE-2025-27475	Windows Update Stack	Elevation of Privilege
Case 87975	Windows Language Experience Service	Information Disclosure

Results

- Special thanks to **prior researchers** and **James Forshaw**





Results

- Analysis of prior work is essential for discovering new vulnerabilities
- Our next step is to automate the discovery of these patterns
- Our Journey isn’t over :D



OFF-BY-ONE
2025

Thank you

COM-pletely Unplanned

: A Windows Bug Hunter’s Journey to LPE

ENKI
WHITEHAT

Jongseong Kim, Dongjun Kim